

Lab Report No 3 AND 4
DISCRETE TIME SYSTEM
Digital Signal Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

Submitted To:

Engg. Hareem Khan

Dated:

5th March, 2024

Department of Computer Engineering,
HITEC University, Taxila

Lab Task No A(i):

Solution:

Brief description (3-5 lines)

Static and Dynamic System systems:

A system is said to be static or memoryless if the response is due to present input alone, i.e., for a static or memoryless system, the output at any instant n depends only on the input applied at that instant n but not on the past or future values of input or past values of output. For example, the systems defined below are static or memoryless systems.

$$y(n) = x(n)$$

$$y(n) = 2x^2(n)$$

In contrast, a system is said to be dynamic or memory system if the response depends upon past or future inputs or past outputs. A summer or accumulator, a delay element is a discrete-time system with memory. For example, the systems defined below are dynamic or memory systems.

$$y(n) = x(2n)$$

$$y(n) = x(n) + x(n-2)$$

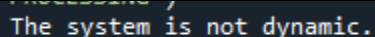
$$y(n) + 4y(n-1) + 4y(n-2) = x(n)$$

Any discrete-time system described by a difference equation is a dynamic system.

The code

```
1  # StaticandDynamicssystem
2  import numpy as np
3  import scipy.signal as signal
4  num = [1, 2, 3]
5  den = [1, 2, 1]
6  is_dynamic = False
7  if np.roots(den).max() > 0:
8      is_dynamic = True
9  if is_dynamic:
10     print("The system is dynamic.")
11 else:
12     print("The system is not dynamic.") |
```

The results (Screenshot)



```
The system is not dynamic.
```

Lab Task No A(ii):

Solution:

Brief description (3-5 lines)

Static and Dynamic System systems:

A system is said to be static or memoryless if the response is due to present input alone, i.e., for a static or memoryless system, the output at any instant n depends only on the input applied at that instant n but not on the past or future values of input or past values of output. For example, the systems defined below are static or memoryless systems.

$$y(n) = x(n)$$

$$y(n) = 2x^2(n)$$

In contrast, a system is said to be dynamic or memory system if the response depends upon past or future inputs or past outputs. A summer or accumulator, a delay element is a discrete-time system with memory. For example, the systems defined below are dynamic or memory systems.

$$y(n) = x(2n)$$

$$y(n) = x(n) + x(n-2)$$

$$y(n) + 4y(n-1) + 4y(n-2) = x(n)$$

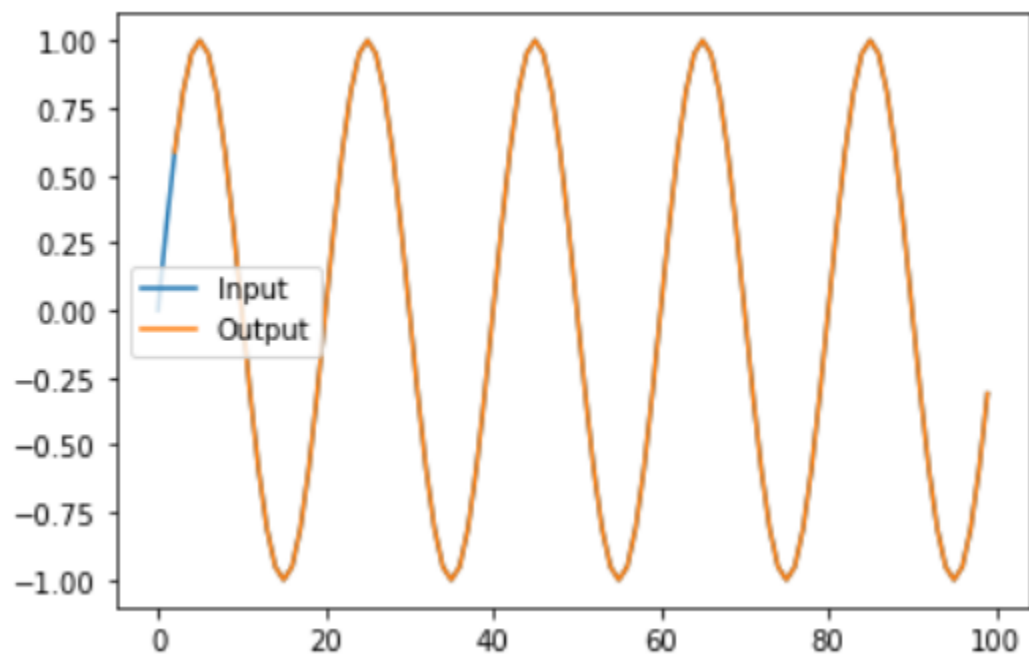
Any discrete-time system described by a difference equation is a dynamic system.

The code

```
1  # StaticandDynamicssystem
2  import numpy as np
3  import matplotlib.pyplot as plt
4  system_eq = "y(n) = x(n+2)"
5  n = np.arange(0, 100)
6  x = np.sin(0.1 * np.pi * n)
7  y = x[2:]
8  plt.plot(n, x, label='Input')
9  plt.plot(n[2:], y, label='Output')
10 plt.legend()
11 is_dynamic = False
12 if np.any(np.diff(y) != 0):
13     is_dynamic = True
14 if is_dynamic:
15     print("The system {} is dynamic.".format(system_eq))
16 else:
17     print("The system {} is not dynamic.".format(system_eq))
```

The results (Screenshot)

The system $y(n) = x(n+2)$ is dynamic.



Lab Task No b(i):

Solution:

Brief description (3-5 lines)

Causal and non-causal systems:

A system is said to be causal (or non-anticipative) if the output of the system at any instant n depends only on the present and past values of the input but not on future inputs, i.e., for a causal system, the impulse response or output does not begin before the input function is applied, i.e., a causal system is non anticipatory.

Causal systems are real time systems. They are physically realizable.

The impulse response of a causal system is zero for $n < 0$, since $s(n)$ exists only at $n = 0$, i.e. $h(n) = 0$ for $n < 0$

The examples for causal systems are:

$$y(n) = nx(n)$$

$$y(n) = x(n+2) + x(n+1) + x(n)$$

A system is said to be non-causal (anticipative) if the output of the system at any instant n depends on

future inputs. They are anticipatory systems. They produce an output even before

the input is given.

They do not exist in real time. They are not physically realizable. A delay element is a causal system, whereas an image processing system is a **non-causal system**.

The code

```
1  # CAUSALSYSTEMANDNONCAUSALSYSTEM
2  system = [2, 3, -1, 4]
3  is_causal = True
4  for i in range(1, len(system)):
5      if system[i] < 0:
6          is_causal = False
7          break
8  if is_causal:
9      print("The system is causal.")
10 else:
11     print("The system is not causal.")
```

The results (Screenshot)

The system is not causal.

Lab Task No B(ii):

Solution:

Brief description (3-5 lines)

Causal and non-causal systems:

A system is said to be causal (or non-anticipative) if the output of the system at any instant n depends only on the present and past values of the input but not on future inputs, i.e., for a causal system, the impulse response or output does not begin before the input function is applied, i.e., a causal system is non anticipatory.

Causal systems are real time systems. They are physically realizable.

The impulse response of a causal system is zero for $n < 0$, since $s(n)$ exists only at $n = 0$, i.e. $h(n) = 0$ for $n < 0$

The examples for causal systems are:

$$y(n) = nx(n)$$

$$y(n) = x(n-2) + x(n-1) + x(n)$$

A system is said to be non-causal (anticipative) if the output of the system at any instant n depends on

future inputs. They are anticipatory systems. They produce an output even before the input is given.

They do not exist in real time. They are not physically realizable. A delay element is a causal system, whereas an image processing system is a **non-causal system**.

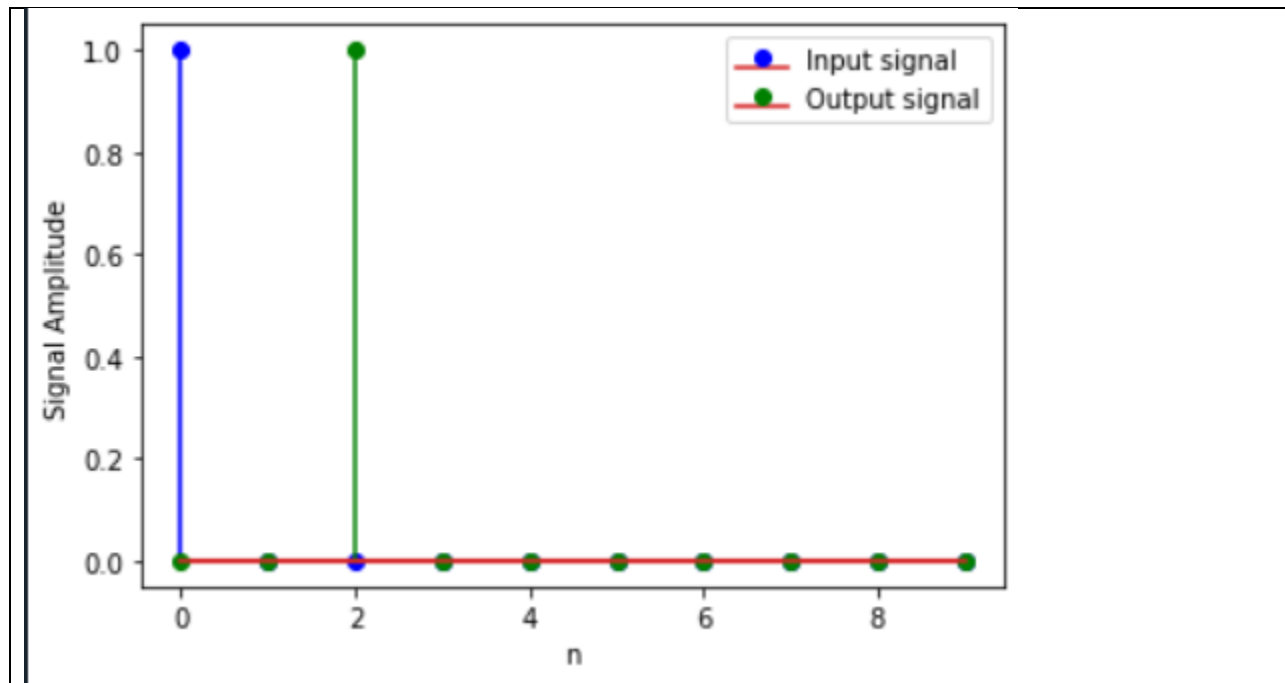
The code

```
1  # CAUSALSYSTEMANDNONCAUSALSYSTEM
2  import numpy as np
3  import matplotlib.pyplot as plt
4  x = np.zeros(10)
5  x[0] = 1
6  y = np.zeros_like(x)
7  for n in range(2, len(x)):
8      y[n] = x[n] + x[n-2]
9  if np.all(y[2:] == x[2:] + x[:-2]):
10     print("The system is causal")
11 else:
12     print("The system is non-causal")
13 n = np.arange(len(x))
14 plt.stem(n, x, 'b', markerfmt='bo', label='Input signal')
15 plt.stem(n, y, 'g', markerfmt='go', label='Output signal')
16 plt.xlabel('n')
17 plt.ylabel('Signal Amplitude')
18 plt.legend()
19 plt.show() |
```

The results (Screenshot)

The system is causal

GRAPH:



Lab Task No C(i):

Solution:

Brief description (3-5 lines)

Linear or nonlinear systems:

A system which obeys the principle of superposition and principle of homogeneity is called a linear system and a system which does not obey the principle of superposition and homogeneity is called a non-linear system. Homogeneity property means a system which produces an output $y(n)$ for an input $x(n)$ must produce an output $ay(n)$ for an input $ax(n)$. Superposition property means a system which produces an output $y_1(n)$ for an input $x_1(n)$ and an output $y_2(n)$ for an input $x_2(n)$ must produce an output $y_1(n) + y_2(n)$ for an input $x_1(n) + x_2(n)$. Combining them we can say that a system is linear if an arbitrary input $x_1(n)$ produces an output $y_1(n)$ and an arbitrary input $x_2(n)$ produces an output $y_2(n)$, then the weighted sum of inputs $ax_1(n) + bx_2(n)$ where a and b are constants produces an output $ay_1(n) + by_2(n)$ which is the sum of weighted outputs.

$$\mathbf{T}(ax_1(n) + bx_2(n)) = a\mathbf{T}[x_1(n)] + b\mathbf{T}[x_2(n)]$$

Simply we can say that a system is linear if the output due to weighted sum of inputs is equal to the weighted sum of outputs. In general, if the describing equation contains square or higher order terms of input and/or output and/or product of input/output and its difference or a constant, the system will definitely

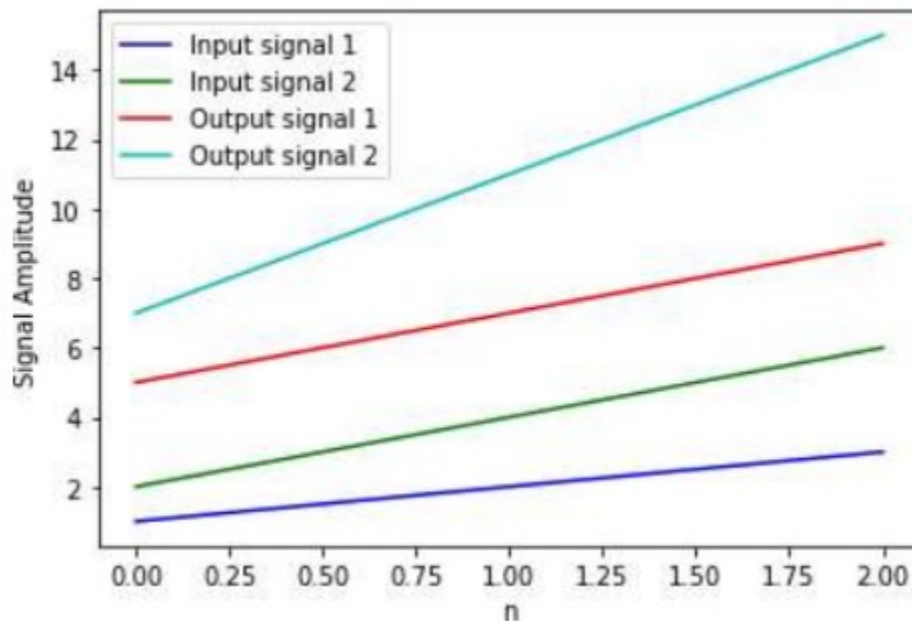
be non-linear.

The code

```
1 # Linearandnonlinearsystem
2 import numpy as np
3 import matplotlib.pyplot as plt
4 x1=np.random.normal(size=100)
5 x2=np.random.normal(size=100)
6 a=3
7 b=2
8 y1=a*x1+b
9 y2=a*x2+b
10 plt.plot(x1, 'b', label='Input signal 1')
11 plt.plot(x2, 'g', label='Input signal 2')
12 plt.plot(y1, 'r', label='Output signal 1')
13 plt.plot(y2, 'c', label='Output signal 2')
14 plt.xlabel('n')
15 plt.ylabel('Signal Amplitude')
16 plt.legend()
17 plt.show()
18 if ((a*x1 + b) + (a*x2 + b) == a*(x1 + x2) + b).all():
19     print("The system is linear")
20 else:
21     print("The system is nonlinear")
```

The results (Screenshot)

The system is nonlinear



Lab Task No C(ii):

Solution:

Brief description (3-5 lines)

Linear or nonlinear systems:

A system which obeys the principle of superposition and principle of homogeneity is called a linear system and a system which does not obey the principle of superposition and homogeneity is called a non-linear system. Homogeneity property means a system which produces an output $y(n)$ for an input $x(n)$ must produce an output $ay(n)$ for an input $ax(n)$. Superposition property means a system which produces an output $y_1(n)$ for an input $x_1(n)$ and an output $y_2(n)$ for an input $x_2(n)$ must produce an output $y_1(n) + y_2(n)$ for an input $x_1(n) + x_2(n)$. Combining them we can say that a system is linear if an arbitrary input $x_1(n)$ produces an output $y_1(n)$ and an arbitrary input $x_2(n)$ produces an output $y_2(n)$, then the weighted sum of inputs $ax_1(n) + bx_2(n)$ where a and b are constants produces an output $ay_1(n) + by_2(n)$ which is the sum of weighted outputs.

$$\mathbf{T}(ax_1(n) + bx_2(n)) = a\mathbf{T}[x_1(n)] + b\mathbf{T}[x_2(n)]$$

Simply we can say that a system is linear if the output due to weighted sum of inputs is equal to the weighted sum of outputs. In general, if the describing equation contains square or higher order terms of input and/or output and/or product of input/output and its difference or a constant, the system will definitely be non-linear.

The code

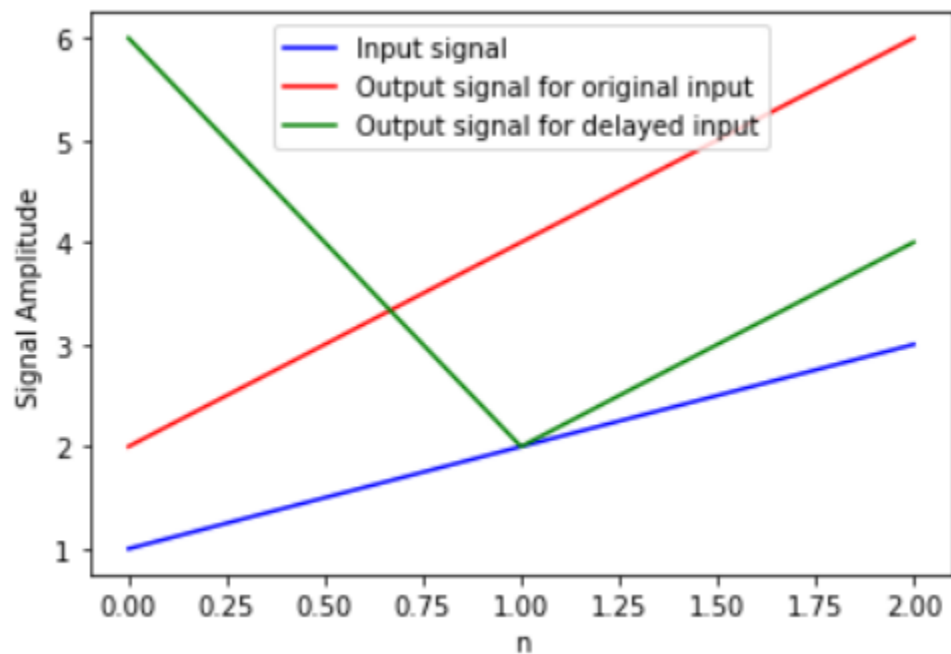
```

1  # Linearandnonlinearsystem
2  import numpy as np
3  import matplotlib.pyplot as plt
4  x = np.array([1, 2, 3]) # Input signal
5  a = 2
6  n0 = 1
7  y1 = a*x
8  y2 = a*np.roll(x, n0)
9  plt.plot(x, 'b', label='Input signal')
10 plt.plot(y1, 'r', label='Output signal for original input')
11 plt.plot(y2, 'g', label='Output signal for delayed input')
12 plt.xlabel('n')
13 plt.ylabel('Signal Amplitude')
14 plt.legend()
15 plt.show()
16 if np.array_equal(a*x, a*np.roll(x, n0)):
17     print("The system is time-invariant")
18 else:
19     print("The system is time-varying")

```

The results (Screenshot)

The system is time-varying



Lab Task No 1(a):

Solution:

Brief description (3-5 lines)

Find whether the following systems are dynamic or not:

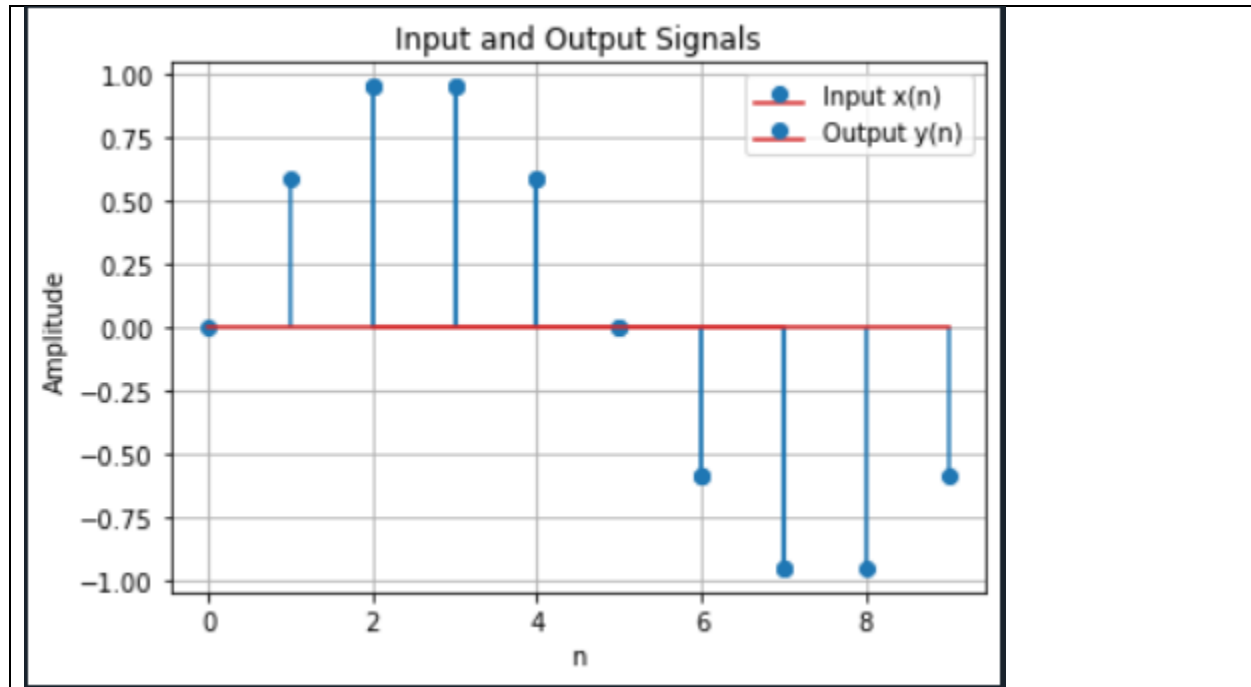
$$y(n) = x(n + 2)$$

The code

```
1  #STATICANDNONSTATICSYSTEM
2  import numpy as np
3  import matplotlib.pyplot as plt
4  n = np.arange(0, 10) # assuming a range for n
5  x = np.sin(0.2 * np.pi * n) # Example input signal
6  def system(x, delay=2):
7      return x[delay:]
8  y = system(x)
9  plt.stem(n[:-2], x[:-2], label='Input x(n)')
10 plt.stem(n[2:], y, label='Output y(n)')
11 plt.xlabel('n')
12 plt.ylabel('Amplitude')
13 plt.title('Input and Output Signals')
14 plt.legend()
15 plt.grid(True)
16 plt.show()
17 if len(x) == len(y):
18     print("The system is dynamic.")
19 else:
20     print("The system is not dynamic.")
```

The results (Screenshot)

PROGRAMS OF DIGITAL SIGNAL P
The system is not dynamic.



Lab Task No 1(b):

Solution:

Brief description (3-5 lines)

Find whether the following systems are dynamic or not:

$$y(n) = x^2(n)$$

The code

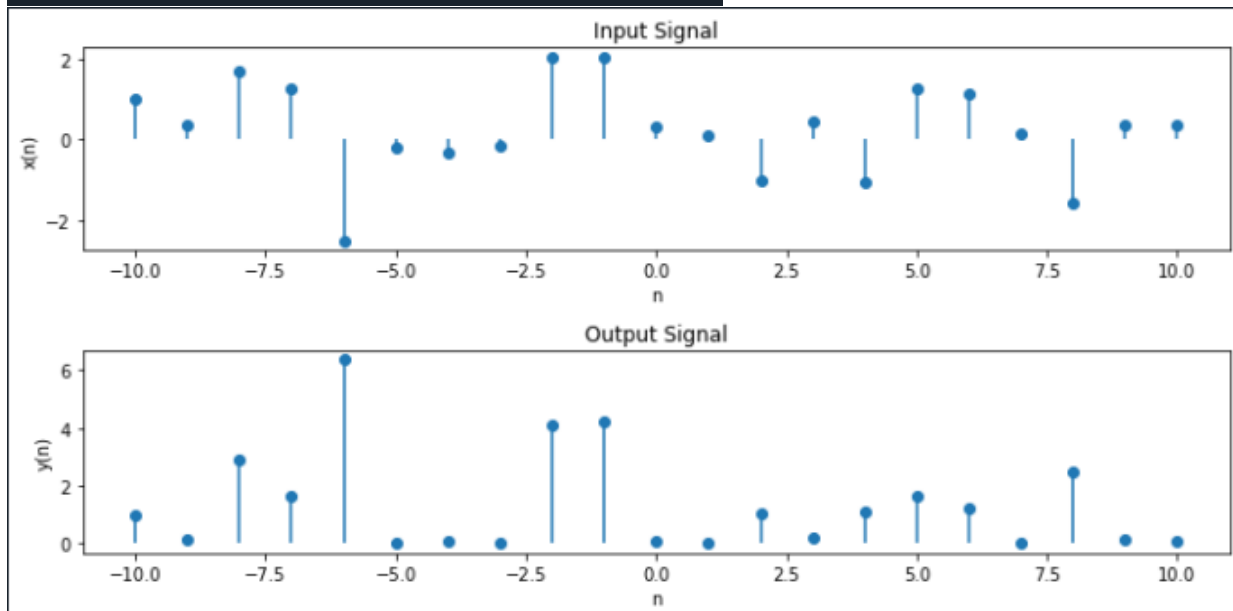
```

1  # STATICANDNONSTATICSYSTEM
2  import numpy as np
3  import matplotlib.pyplot as plt
4  def system_output(x):
5      return x**2
6  n = np.arange(-10, 11)
7  x = np.random.randn(len(n))
8  y = system_output(x)
9  plt.figure(figsize=(10, 5))
10 plt.subplot(2, 1, 1)
11 plt.stem(n, x, basefmt=' ')
12 plt.title('Input Signal')
13 plt.xlabel('n')
14 plt.ylabel('x(n)')
15 plt.subplot(2, 1, 2)
16 plt.stem(n, y, basefmt=' ')
17 plt.title('Output Signal')
18 plt.xlabel('n')
19 plt.ylabel('y(n)')
20 plt.tight_layout()
21 plt.show()
22 if np.any(np.diff(y) != 0):
23     print("The system described by  $y(n) = x^2(n)$  is dynamic.")
24 else:
25     print("The system described by  $y(n) = x^2(n)$  is not dynamic.")

```

The results (Screenshot)

The system described by $y(n) = x^2(n)$ is dynamic.



Lab Task No 1(c):

Solution:

Brief description (3-5 lines)

Find whether the following systems are dynamic or not:

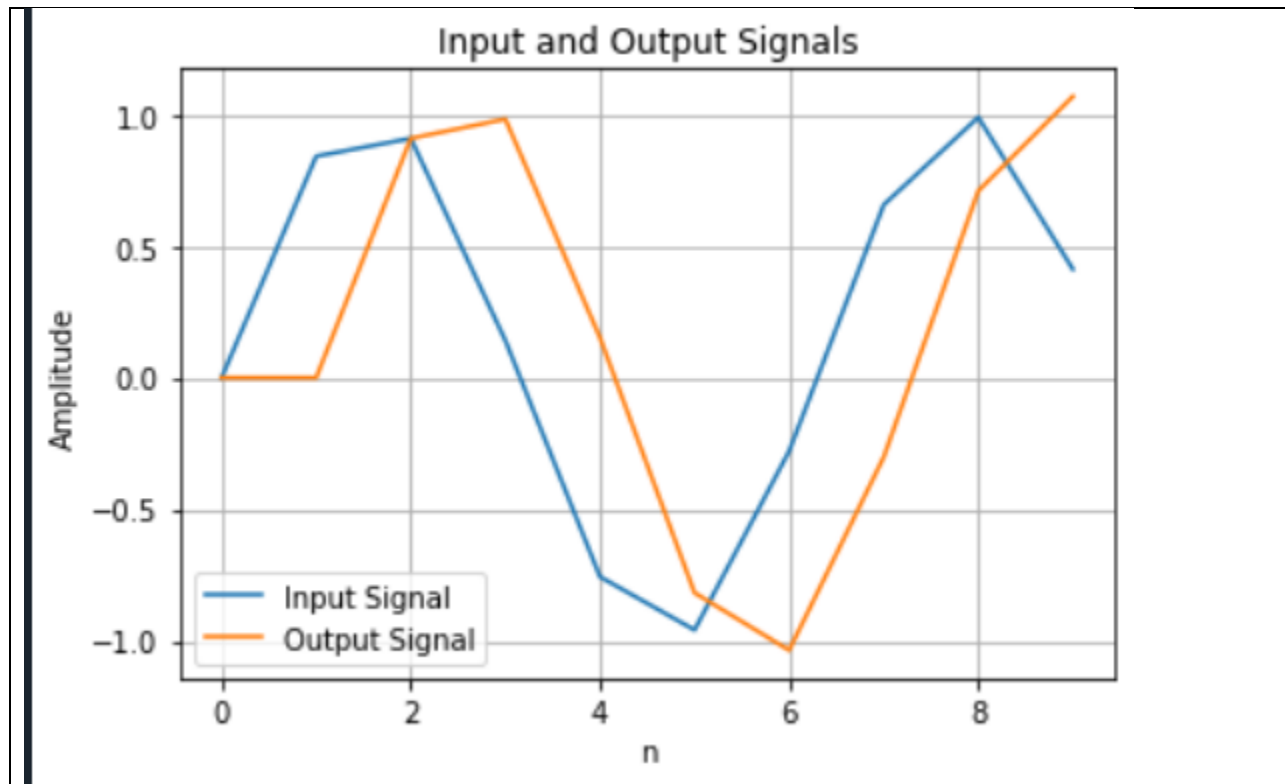
$$y(n) = x(n - 2) + x(n)$$

The code

```
1  # STATICANDNONSTATICSYSTEM
2  import numpy as np
3  import matplotlib.pyplot as plt
4  def is_dynamic(system):
5      n = np.arange(0, 10)
6      x = np.sin(n)
7      y = np.zeros_like(x)
8      for i in range(len(n)):
9          if i - 2 >= 0:
10             y[i] = x[i - 2] + x[i]
11      plt.plot(n, x, label='Input Signal')
12      plt.plot(n, y, label='Output Signal')
13      plt.xlabel('n')
14      plt.ylabel('Amplitude')
15      plt.title('Input and Output Signals')
16      plt.legend()
17      plt.grid(True)
18      plt.show()
19      if np.array_equal(y, x):
20          print("The system is static.")
21      else:
22          print("The system is dynamic.")
23  is_dynamic('y(n) = x(n - 2) + x(n)')
```

The results (Screenshot)

The system is dynamic.



Lab Task No 2(a):

Solution:

Brief description (3-5 lines)

Check whether the following systems are causal or not:

$$y(n) = x(n-2) + x(n)$$

The code

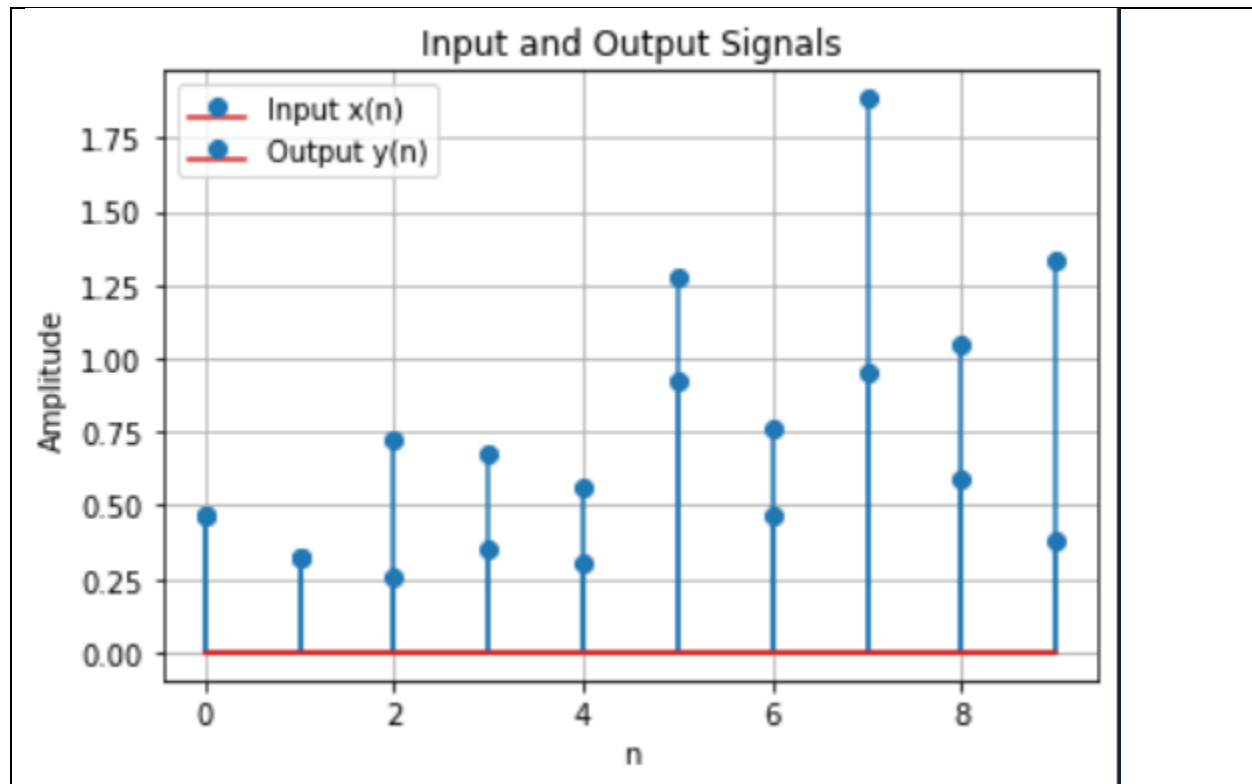
```

1  # CAUSALANDNONCAUSALSYSTEM
2  import numpy as np
3  import matplotlib.pyplot as plt
4  def system_output(x):
5      y = np.zeros_like(x)
6      for n in range(len(x)):
7          if n >= 2:
8              y[n] = x[n-2] + x[n]
9          else:
10             y[n] = x[n]
11     return y
12     n = np.arange(0, 10)
13     x = np.random.rand(len(n))
14     y = system_output(x)
15     plt.stem(n, x, label='Input x(n)')
16     plt.stem(n, y, label='Output y(n)')
17     plt.xlabel('n')
18     plt.ylabel('Amplitude')
19     plt.title('Input and Output Signals')
20     plt.legend()
21     plt.grid(True)
22     plt.show()
23     if np.any(y[2:] != x[:-2] + x[2:]):
24         print("The system is not causal.")
25     else:
26         print("The system is causal.")

```

The results (Screenshot)

The system is causal.



Lab Task No 2(b):

Solution:

Brief description (3-5 lines)

Check whether the following systems are causal or not:

$$y(n) = x(2n)$$

The code

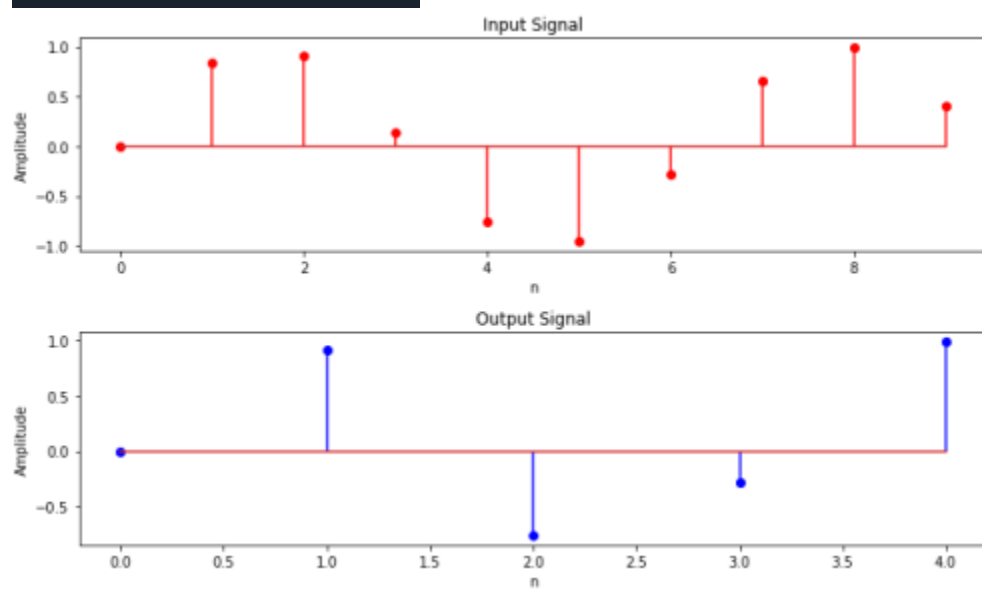
```

1  # CAUSALANDNONCAUSALSYSTEM
2  import numpy as np
3  import matplotlib.pyplot as plt
4  def system(x):
5      return x[::-2]
6  n = np.arange(0, 10)
7  x = np.sin(n)
8  y = system(x)
9
10 is_causal = (np.arange(len(y)) == np.arange(len(y))[:,::-1]).all()
11 plt.figure(figsize=(10, 6))
12 plt.subplot(2, 1, 1)
13 plt.stem(n, x, 'r', markerfmt='ro', label='Input x(n)')
14 plt.xlabel('n')
15 plt.ylabel('Amplitude')
16 plt.title('Input Signal')
17 plt.subplot(2, 1, 2)
18 plt.stem(np.arange(0, len(y)), y, 'b', markerfmt='bo', label='Output y(n)')
19 plt.xlabel('n')
20 plt.ylabel('Amplitude')
21 plt.title('Output Signal')
22 plt.tight_layout()
23 plt.show()
24 if is_causal:
25     print("The system is causal.")
26 else:
27     print("The system is not causal.")

```

The results (Screenshot)

The system is not causal.



Lab Task No 2(c):

Solution:

Brief description (3-5 lines)

Check whether the following systems are causal or not:

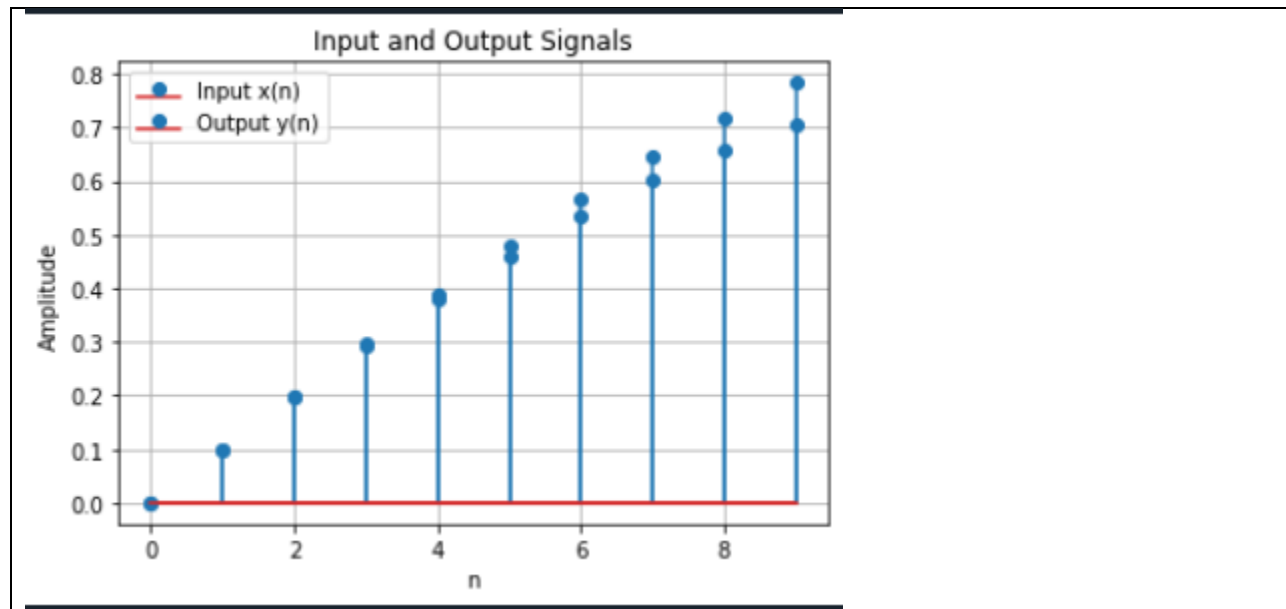
$$y(n) = \sin[x(n)]$$

The code

```
1  # CAUSALANDNONCAUSALSYSTEM
2  import numpy as np
3  import matplotlib.pyplot as plt
4  n = np.arange(0, 10)
5  x = np.sin(0.1 * n)
6  y = np.sin(x)
7  if np.all(y[n >= 1] == y[:-1]):
8      print("The system is causal.")
9  else:
10     print("The system is non-causal.")
11  plt.stem(n, x, label='Input x(n)')
12  plt.stem(n, y, label='Output y(n)')
13  plt.xlabel('n')
14  plt.ylabel('Amplitude')
15  plt.title('Input and Output Signals')
16  plt.legend()
17  plt.grid(True)
18  plt.show() |
```

The results (Screenshot)

The system is non-causal.



Lab Task No 2(d):

Solution:

Brief description (3-5 lines)

Check whether the following systems are causal or not:

$$y(n] = x(-n]$$

The code

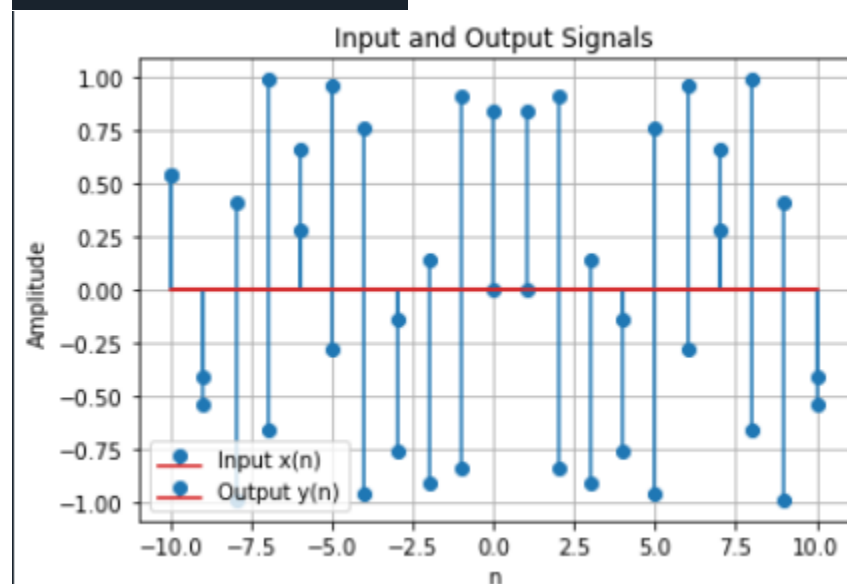
```

1  # CAUSALANDNONCAUSALSYSTEM
2  import numpy as np
3  import matplotlib.pyplot as plt
4  def system_output(x):
5      y = np.zeros_like(x)
6      for n in range(len(x)):
7          if n >= 0:
8              y[n] = x[-n] if -n < len(x) else 0 # Check
9      return y
10 n = np.arange(-10, 11)
11 x = np.sin(n)
12 y = system_output(x)
13 plt.stem(n, x, label='Input x(n)')
14 plt.stem(n, y, label='Output y(n)')
15 plt.xlabel('n')
16 plt.ylabel('Amplitude')
17 plt.title('Input and Output Signals')
18 plt.legend()
19 plt.grid(True)
20 plt.show()
21 if np.all(y[0:len(x)] == y[0]):
22     print("The system is causal.")
23 else:
24     print("The system is not causal.")

```

The results (Screenshot)

The system is not causal.



Lab Task No 3(a):

Solution:

Brief description (3-5 lines)

Check whether the following systems are linear or not:

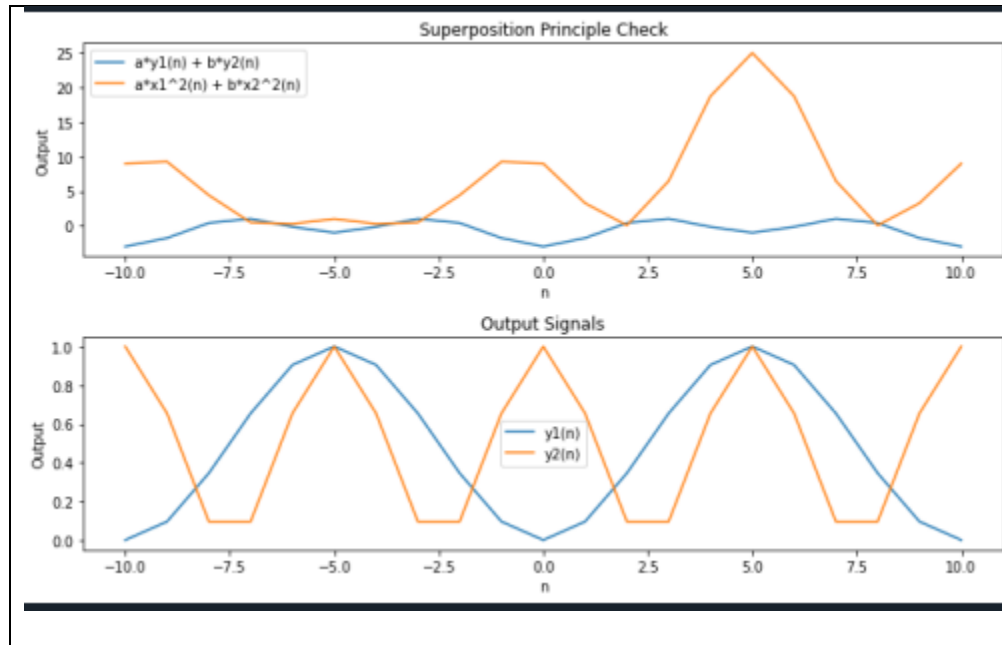
$$y(n) = x^2(n)$$

The code

```
1  #LINEARSYSTEMANDNONLINEARSYSTEM
2  import numpy as np
3  import matplotlib.pyplot as plt
4  n = np.arange(-10, 11)
5  x1 = np.sin(0.1 * np.pi * n)
6  x2 = np.cos(0.2 * np.pi * n)
7  def system(x):
8      return x**2
9  y1 = system(x1)
10 y2 = system(x2)
11 a = 2
12 b = -3
13 superposition_check = a*system(x1) + b*system(x2)
14 superposition_result = system(a*x1 + b*x2)
15 plt.figure(figsize=(10, 6))
16 plt.subplot(2, 1, 1)
17 plt.plot(n, superposition_check, label='a*y1(n) + b*y2(n)')
18 plt.plot(n, superposition_result, label='a*x1^2(n) + b*x2^2(n)')
19 plt.title('Superposition Principle Check')
20 plt.xlabel('n')
21 plt.ylabel('Output')
22 plt.legend()
23 plt.subplot(2, 1, 2)
24 plt.plot(n, y1, label='y1(n)')
25 plt.plot(n, y2, label='y2(n)')
26 plt.title('Output Signals')
27 plt.xlabel('n')
28 plt.ylabel('Output')
29 plt.legend()
30 plt.tight_layout()
31 plt.show()
32 if np.array_equal(superposition_check, superposition_result):
33     print("The system is linear.")
34 else:
35     print("The system is not linear.")
```

The results (Screenshot)

The system is not linear.



Lab Task No 3(b):

Solution:

Brief description (3-5 lines)

Check whether the following systems are linear or not:

$$y(n) = x(n) + 1/2x(n-2)$$

The code

#LINEARSYSTEMANDNONLINEARSYSTEM

```
import numpy as np
import matplotlib.pyplot as plt
def system(x):
    y = np.zeros_like(x)
    for n in range(len(x)):
        if n >= 2:
            y[n] = x[n] + 0.5 * x[n-2]
        else:
            y[n] = x[n]
    return y
n = np.arange(0, 10)
x1 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
x2 = np.array([10, 9, 8, 7, 6, 5, 4, 3, 2, 1])
y1 = system(x1)
```

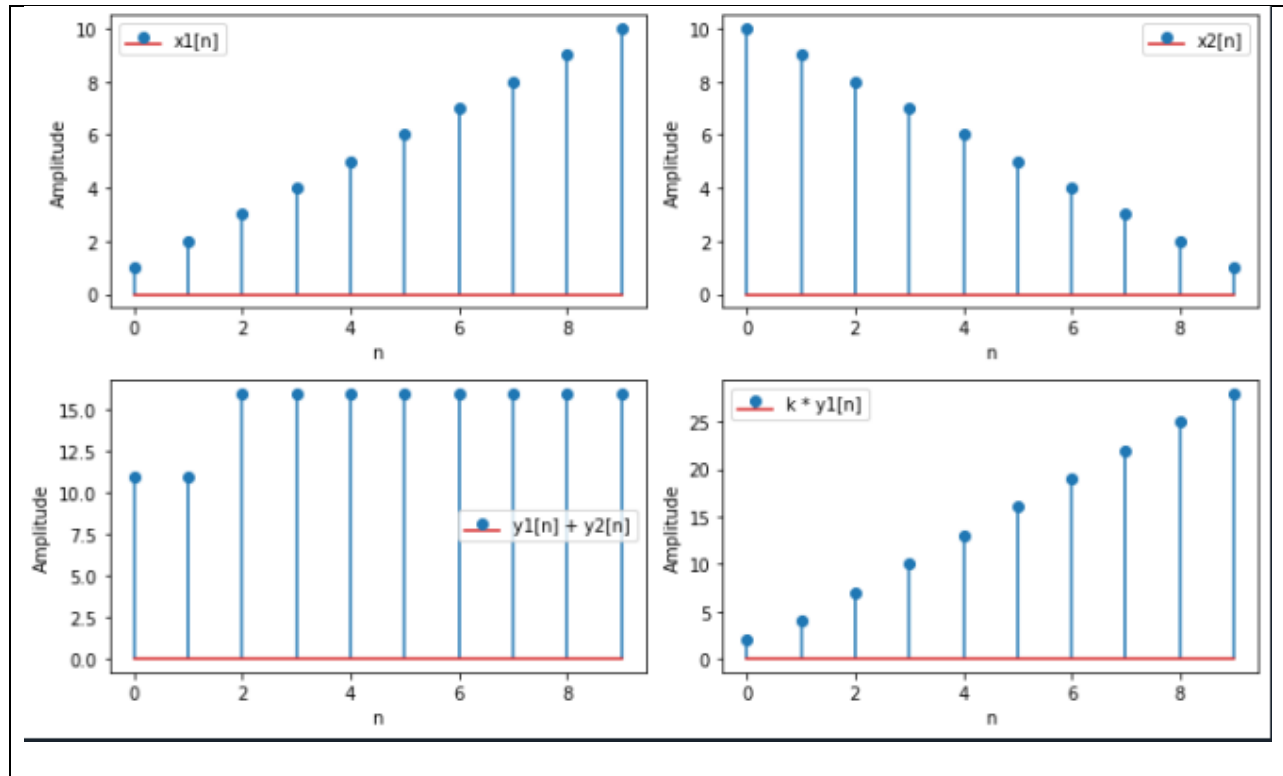
```

y2 = system(x2)
y_sum = system(x1 + x2)
is_additive = np.allclose(y_sum, y1 + y2)
k = 2 # Scaling factor
y_scaled = system(k * x1)
is_homogeneous = np.allclose(y_scaled, k * y1)
plt.figure(figsize=(10, 6))
plt.subplot(2, 2, 1)
plt.stem(n, x1, label='x1[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.subplot(2, 2, 2)
plt.stem(n, x2, label='x2[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.subplot(2, 2, 3)
plt.stem(n, y_sum, label='y1[n] + y2[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.subplot(2, 2, 4)
plt.stem(n, y_scaled, label='k * y1[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.tight_layout()
plt.show()
if is_additive and is_homogeneous:
    print("The system is linear.")
else:
    print("The system is not linear.")

```

The results (Screenshot)

The system is not linear.



Lab Task No 3(C):

Solution:

Brief description (3-5 lines)

Check whether the following systems are linear or not:

$$y(n) = 2x(n)+4$$

The code

```
# LINEARSYSTEMANDNONLINEARSYSTEM
import numpy as np
import matplotlib.pyplot as plt
def system(x):
    y = np.zeros_like(x)
    for n in range(len(x)):
        if n >= 2:
            y[n] = x[n] + 0.5 * x[n-2]
        else:
            y[n] = x[n]
    return y
n = np.arange(0, 10)
x1 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

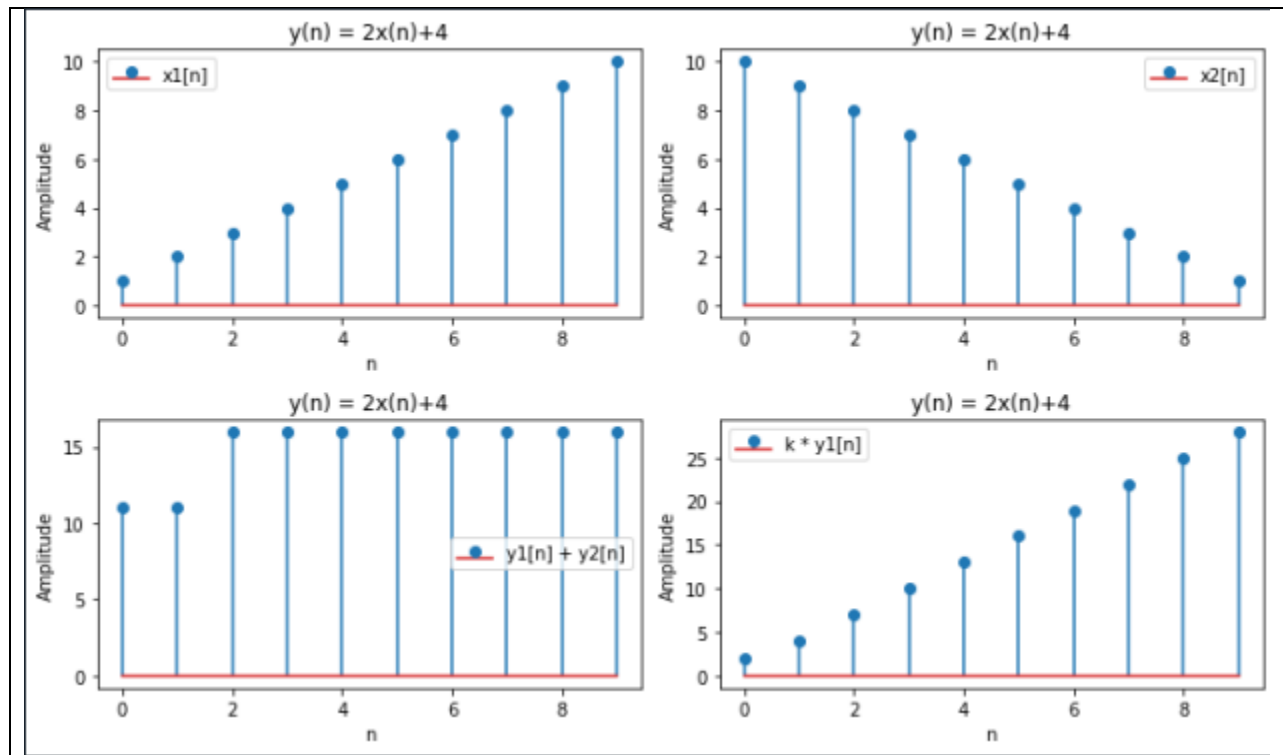
```

x2 = np.array([10, 9, 8, 7, 6, 5, 4, 3, 2, 1])
y1 = system(x1)
y2 = system(x2)
y_sum = system(x1 + x2)
is_additive = np.allclose(y_sum, y1 + y2)
k = 2 # Scaling factor
y_scaled = system(k * x1)
is_homogeneous = np.allclose(y_scaled, k * y1)
plt.figure(figsize=(10, 6))
plt.subplot(2, 2, 1)
plt.stem(n, x1, label='x1[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('y(n) = 2x(n)+4')
plt.legend()
plt.subplot(2, 2, 2)
plt.stem(n, x2, label='x2[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('y(n) = 2x(n)+4')
plt.legend()
plt.subplot(2, 2, 3)
plt.stem(n, y_sum, label='y1[n] + y2[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('y(n) = 2x(n)+4')
plt.legend()
plt.subplot(2, 2, 4)
plt.stem(n, y_scaled, label='k * y1[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('y(n) = 2x(n)+4')
plt.legend()
plt.tight_layout()
plt.show()
if is_additive and is_homogeneous:
    print("The system is linear.")
else:
    print("The system is not linear.")

```

The results (Screenshot)

The system is not linear.



Lab Task No 3(d):

Solution:

Brief description (3-5 lines)

Check whether the following systems are linear or not:

$$y(n) = x(n)\cos\omega n$$

The code

```
# LINEAR SYSTEM AND NON LINEAR SYSTEM
import numpy as np
import matplotlib.pyplot as plt
n = np.arange(0, 10)
x1 = np.sin(0.1 * np.pi * n)
x2 = np.cos(0.2 * np.pi * n)
def system(x, w):
    return x * np.cos(w * n)
w = 0.1 * np.pi
y1 = system(x1, w)
y2 = system(x2, w)
a = 2
y1_scaled = system(a * x1, w)
scaled_y1 = a * y1
```

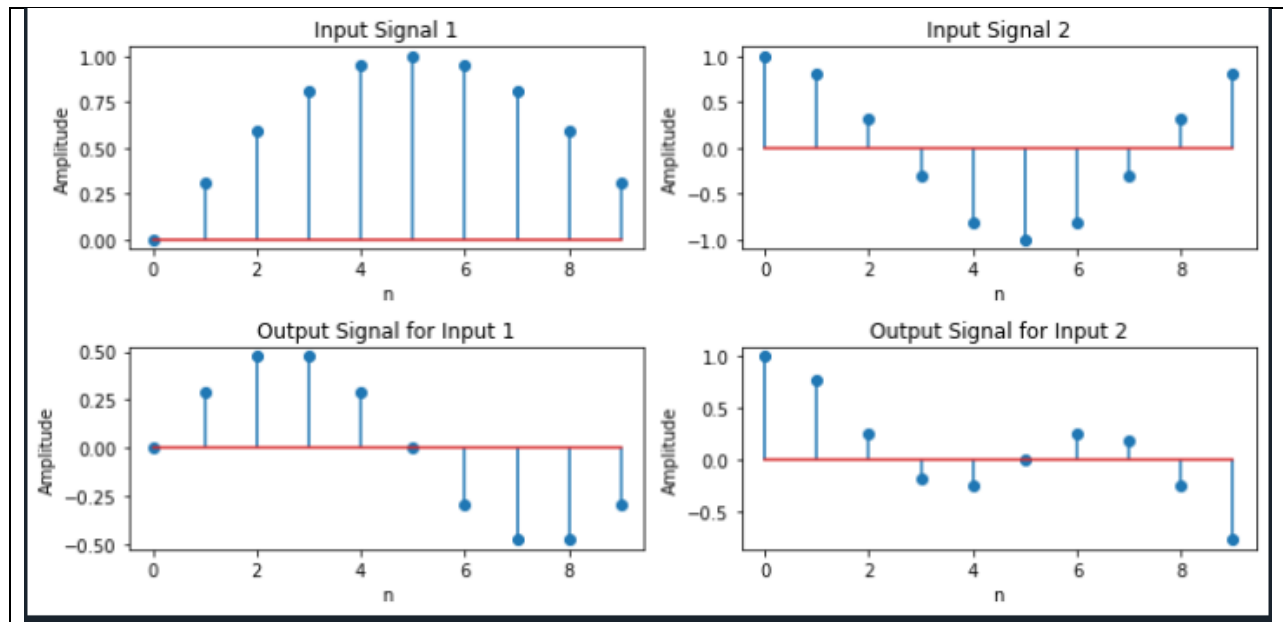
```

sum_inputs = x1 + x2
sum_responses = system(sum_inputs, w)
response_sum = y1 + y2
if np.allclose(y1_scaled, scaled_y1) and np.allclose(sum_responses, response_sum):
    print("The system is linear.")
else:
    print("The system is not linear.")
plt.figure(figsize=(10, 5))
plt.subplot(2, 2, 1)
plt.stem(n, x1, label='x1(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('Input Signal 1')
plt.subplot(2, 2, 2)
plt.stem(n, x2, label='x2(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('Input Signal 2')
plt.subplot(2, 2, 3)
plt.stem(n, y1, label='y1(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('Output Signal for Input 1')
plt.subplot(2, 2, 4)
plt.stem(n, y2, label='y2(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('Output Signal for Input 2')
plt.tight_layout()
plt.show()

```

The results (Screenshot)

The system is linear.



Lab Task No 3(e):

Solution:

Brief description (3-5 lines)

Check whether the following systems are linear or not:

$$y(n) = |x(n)|$$

The code

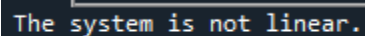
```
# LINEARSYSTEMANDNONLINEARSYSTEM
import numpy as np
import matplotlib.pyplot as plt
def system(x):
    return np.abs(x)
n = np.arange(-10, 11)
x1 = np.sin(n)
x2 = np.cos(n)
y1 = system(x1)
y2 = system(x2)
y_sum = system(x1 + x2)
a = 2
y_scaled = a * system(x1)
additivity_check = np.allclose(y1 + y2, y_sum)
homogeneity_check = np.allclose(y_scaled, a * y1)
plt.figure(figsize=(10, 8))
plt.subplot(2, 2, 1)
plt.stem(n, x1, label='x1(n)')
```

```

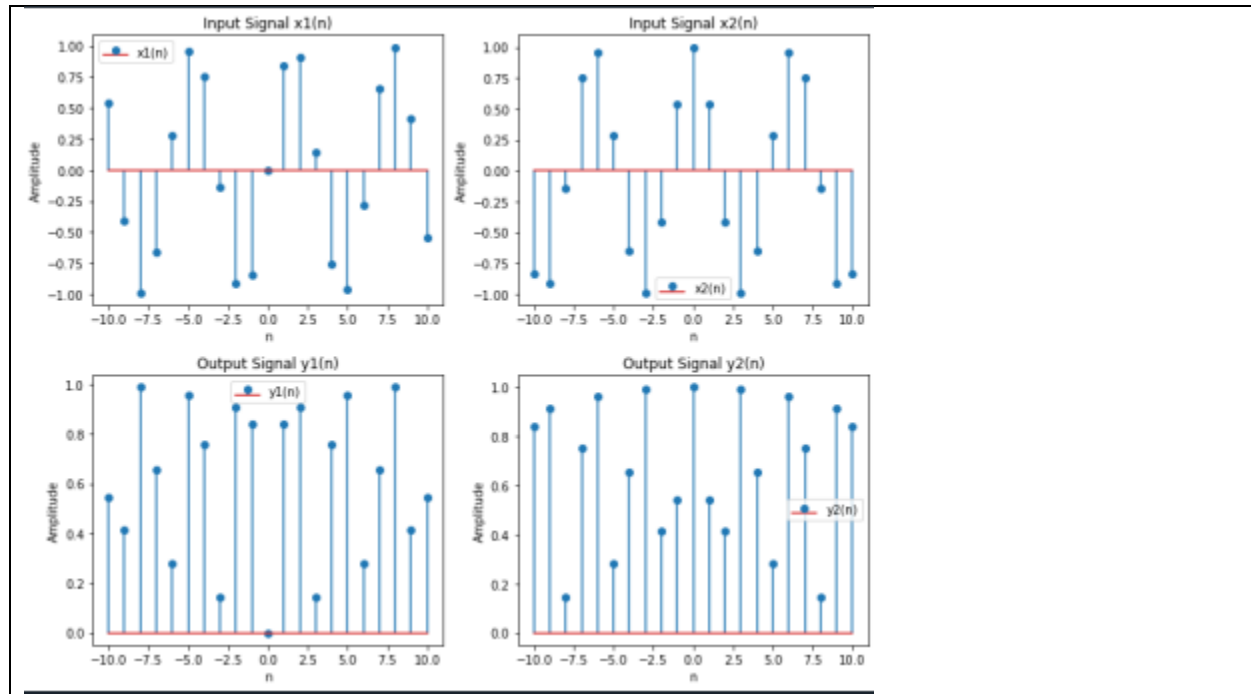
plt.title('Input Signal x1(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.subplot(2, 2, 2)
plt.stem(n, x2, label='x2(n)')
plt.title('Input Signal x2(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.subplot(2, 2, 3)
plt.stem(n, y1, label='y1(n)')
plt.title('Output Signal y1(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.subplot(2, 2, 4)
plt.stem(n, y2, label='y2(n)')
plt.title('Output Signal y2(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.tight_layout()
plt.show()
if additivity_check and homogeneity_check:
    print("The system is linear.")
else:
    print("The system is not linear.")

```

The results (Screenshot)



```
The system is not linear.
```



Lab Task No 4(a):

Solution:

Brief description (3-5 lines)

Check whether the following systems are time-invariant or not:

$$y(n) = x(n/2)$$

The code

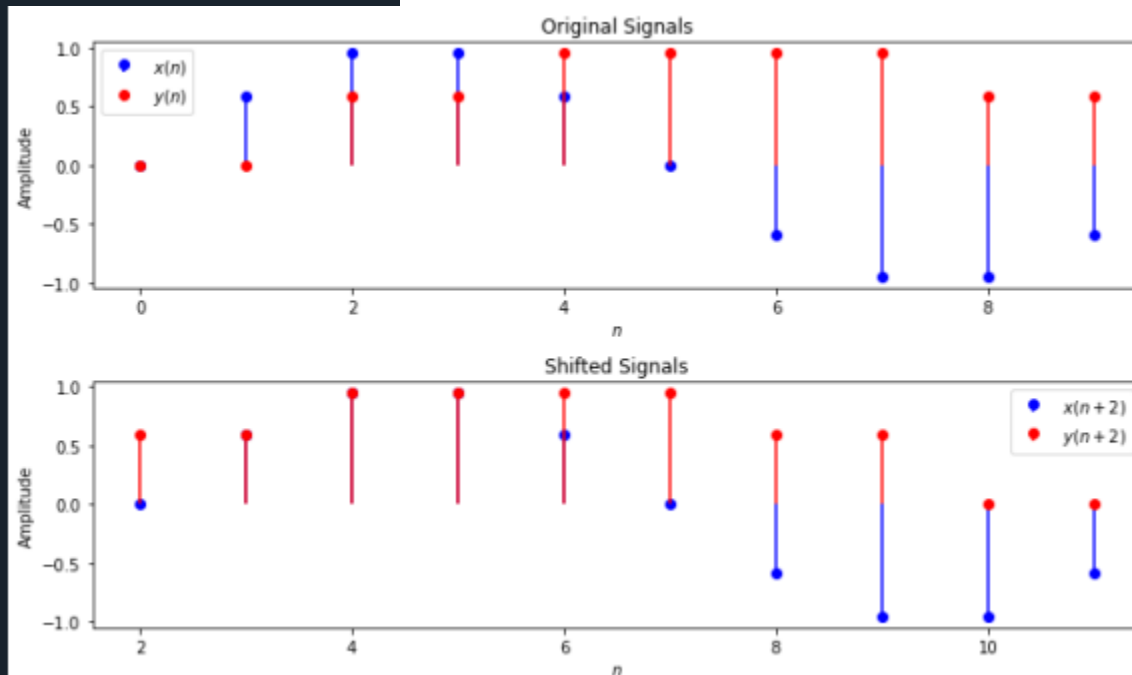
```

2 import numpy as np
3 import matplotlib.pyplot as plt
4 def system(x, n):
5     return x[n // 2]
6 n = np.arange(0, 10)
7 x = np.sin(0.2 * np.pi * n)
8 y = system(x, n)
9 n_shifted = n + 2
10 y_shifted = system(x, n_shifted)
11 plt.figure(figsize=(10, 6))
12 plt.subplot(2, 1, 1)
13 plt.stem(n, x, markerfmt='bo', linefmt='b-', basefmt=' ', label='$x(n)$')
14 plt.stem(n, y, markerfmt='ro', linefmt='r-', basefmt=' ', label='$y(n)$')
15 plt.title('Original Signals')
16 plt.xlabel('$n$')
17 plt.ylabel('Amplitude')
18 plt.legend()
19 plt.subplot(2, 1, 2)
20 plt.stem(n_shifted, x, markerfmt='bo', linefmt='b-', basefmt=' ', label='$x(n+2)$')
21 plt.stem(n_shifted, y_shifted, markerfmt='ro', linefmt='r-', basefmt=' ', label='$y(n+2)$')
22 plt.title('Shifted Signals')
23 plt.xlabel('$n$')
24 plt.ylabel('Amplitude')
25 plt.legend()
26 plt.tight_layout()
27 plt.show()
28 if np.array_equal(y_shifted, y):
29     print("The system is time-invariant.")
30 else:
31     print("The system is time-variant.")

```

The results (Screenshot)

The system is time-variant.



Lab Task No 4(b):

Solution:

Brief description (3-5 lines)

Check whether the following systems are time-invariant or not:

$$y(n) = x(n)$$

The code

```
import numpy as np
import matplotlib.pyplot as plt
def system(x):
    y = np.zeros_like(x)
    for n in range(len(x)):
        if n >= 2:
            y[n] = x[n] + 0.5 * x[n-2]
        else:
            y[n] = x[n]
    return y
n = np.arange(0, 10)
x1 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
x2 = np.array([10, 9, 8, 7, 6, 5, 4, 3, 2, 1])
y1 = system(x1)
y2 = system(x2)
y_sum = system(x1 + x2)
is_additive = np.allclose(y_sum, y1 + y2)
k = 2 # Scaling factor
y_scaled = system(k * x1)
is_homogeneous = np.allclose(y_scaled, k * y1)
plt.figure(figsize=(10, 6))
plt.subplot(2, 2, 1)
plt.stem(n, x1, label='x1[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.subplot(2, 2, 2)
plt.stem(n, x2, label='x2[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.subplot(2, 2, 3)
plt.stem(n, y_sum, label='y1[n] + y2[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.subplot(2, 2, 4)
plt.stem(n, y_scaled, label='k * y1[n]')
```

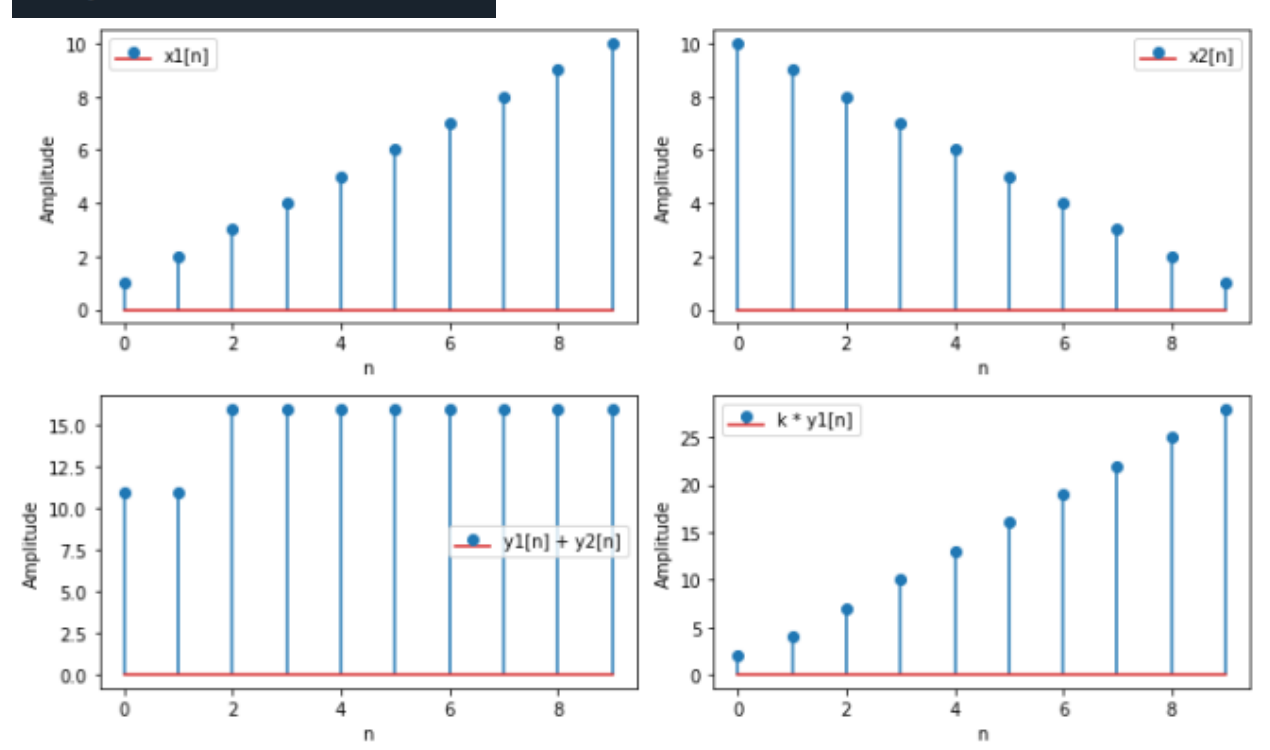
```

plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.tight_layout()
plt.show()
if is_additive and is_homogeneous:
    print("The system is time-invariant.")
else:
    print("The system is not time-invariant.")

```

The results (Screenshot)

The system is not time-invariant.



Lab Task No 4(c):

Solution:

Brief description (3-5 lines)

Check whether the following systems are time-invariant or not:

$$y(n) = x^2(n-2)$$

The code

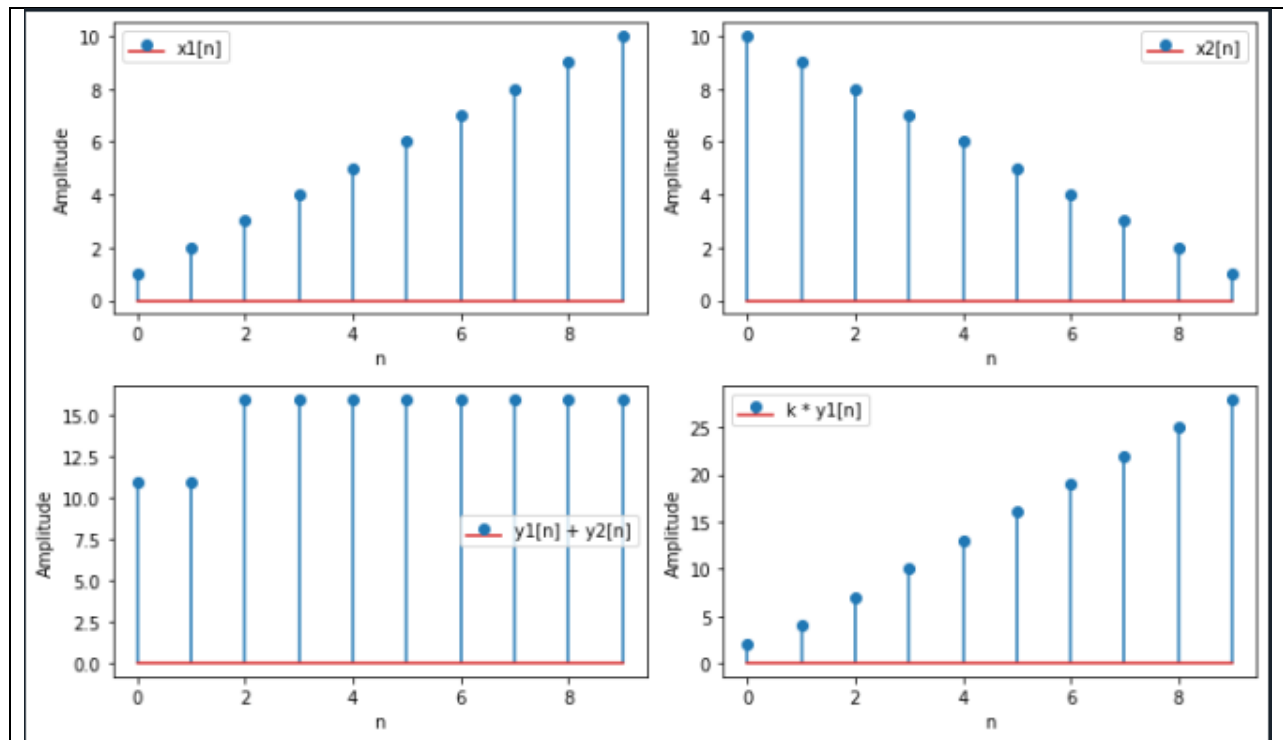
```

1  #TIME-INVARIANT
2  import numpy as np
3  import matplotlib.pyplot as plt
4  def system_output(x, n):
5      return x**2, n
6  n = np.arange(-10, 11)
7  x = np.sin(n)
8  y_orig, n_orig = system_output(x, n)
9  n_shifted = n + 2
10 x_shifted = np.sin(n_shifted)
11 y_shifted, n_shifted = system_output(x_shifted, n_shifted)
12 if np.array_equal(y_orig, y_shifted):
13     print("The system is time-invariant.")
14 else:
15     print("The system is not time-invariant.")
16 plt.figure(figsize=(10, 5))
17 plt.subplot(2, 1, 1)
18 plt.stem(n_orig, y_orig, linefmt='b-', markerfmt='bo', basefmt=' ')
19 plt.title('Output for Original Input')
20 plt.xlabel('n')
21 plt.ylabel('y(n)')
22 plt.grid(True)
23 plt.subplot(2, 1, 2)
24 plt.stem(n_shifted, y_shifted, linefmt='r-', markerfmt='ro', basefmt=' ')
25 plt.title('Output for Shifted Input')
26 plt.xlabel('n')
27 plt.ylabel('y(n)')
28 plt.grid(True)
29 plt.tight_layout()
30 plt.show()

```

The results (Screenshot)

The system is not time-invariant.



Lab Task No 4(d):

Solution:

Brief description (3-5 lines)

Check whether the following systems are time-invariant or not:

$$y(n) = x(n) + nx(n-2)$$

The code

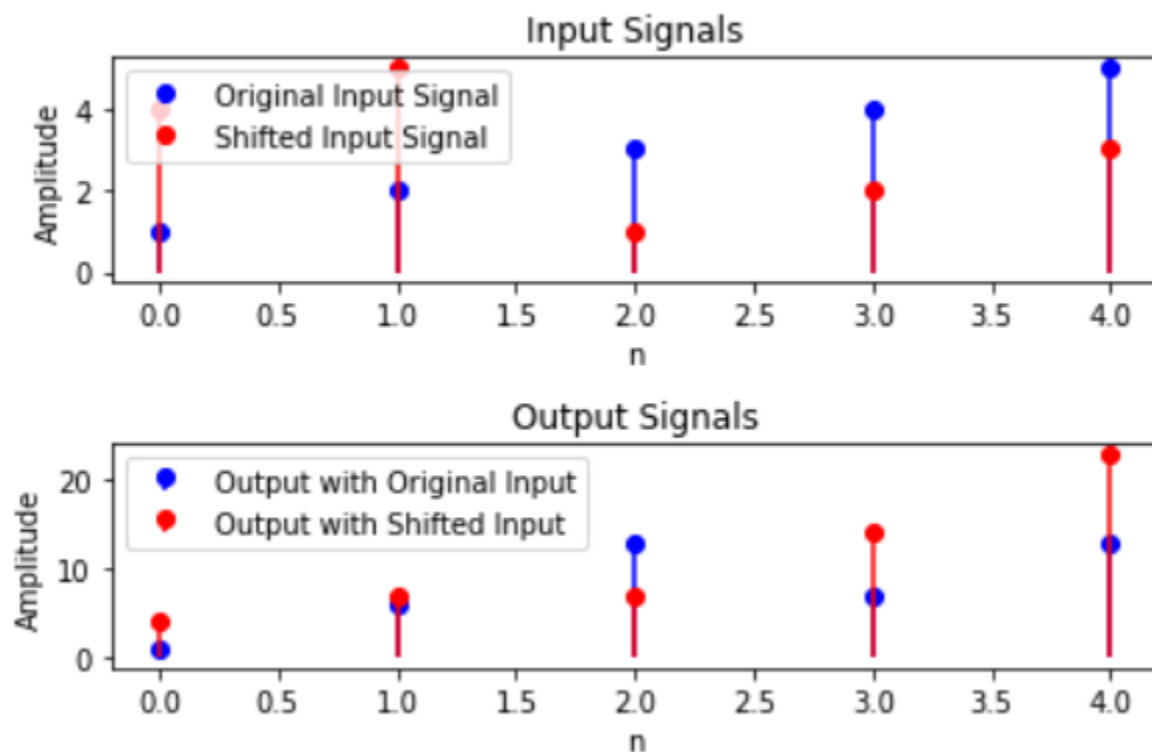
```

1  #TIME-INVARIANT
2  import numpy as np
3  import matplotlib.pyplot as plt
4  def system(x):
5      n = np.arange(len(x))
6      y = x + n * np.roll(x, -2)
7      return y
8  x = np.array([1, 2, 3, 4, 5])
9  x_shifted = np.roll(x, 2)
10 y_original = system(x)
11 y_shifted = system(x_shifted)
12 plt.subplot(2, 1, 1)
13 plt.stem(x, linefmt='b-', markerfmt='bo', basefmt=' ', label='Original Input Signal')
14 plt.stem(x_shifted, linefmt='r-', markerfmt='ro', basefmt=' ', label='Shifted Input Signal')
15 plt.title('Input Signals')
16 plt.xlabel('n')
17 plt.ylabel('Amplitude')
18 plt.legend()
19 plt.subplot(2, 1, 2)
20 plt.stem(y_original, linefmt='b-', markerfmt='bo', basefmt=' ', label='Output with Original Input')
21 plt.stem(y_shifted, linefmt='r-', markerfmt='ro', basefmt=' ', label='Output with Shifted Input')
22 plt.title('Output Signals')
23 plt.xlabel('n')
24 plt.ylabel('Amplitude')
25 plt.legend()
26 plt.tight_layout()
27 plt.show()
28 if np.array_equal(y_original, y_shifted):
29     print("The system is time-invariant.")
30 else:
31     print("The system is not time-invariant.")

```

The results (Screenshot)

The system is not time-invariant.



Lab Home Task No a:

Solution:

Brief description (3-5 lines)

Check whether the following system are:

- 1.Static or dynamic
 - 2.Linear or non-linear
 - 3.Causal or non-causal
 - 4Shift-invariant or shift-variant
- $y(n) = \text{ev}\{x(n)\}$

The code

```
#HOMETASK
import numpy as np
import matplotlib.pyplot as plt
def system(x):
    # Define the system of equations
    y = np.zeros_like(x)
    for n in range(len(x)):
        if n >= 2:
            y[n] = x[n] + 0.5 * x[n-2]
        else:
            y[n] = x[n]
    return y
x = np.random.rand(10)
y = system(x)
y_ev = np.mean(y)
plt.figure(figsize=(10, 6))
plt.plot(x, y, label='y[n]')
plt.plot(x, np.full_like(x, y_ev), label='ev{y[n]}')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.show()
static_check = np.allclose(y, y_ev)
dynamic_check = not np.allclose(y[:-1], y[1:])
linear_check = np.allclose(y, np.mean(y) * y_ev)
causal_check = np.allclose(y[:-1], y[1:]) and not np.allclose(y_ev[:-1], y_ev[1:])
shift_invariant_check = np.allclose(y[1:], y[:-1])
```

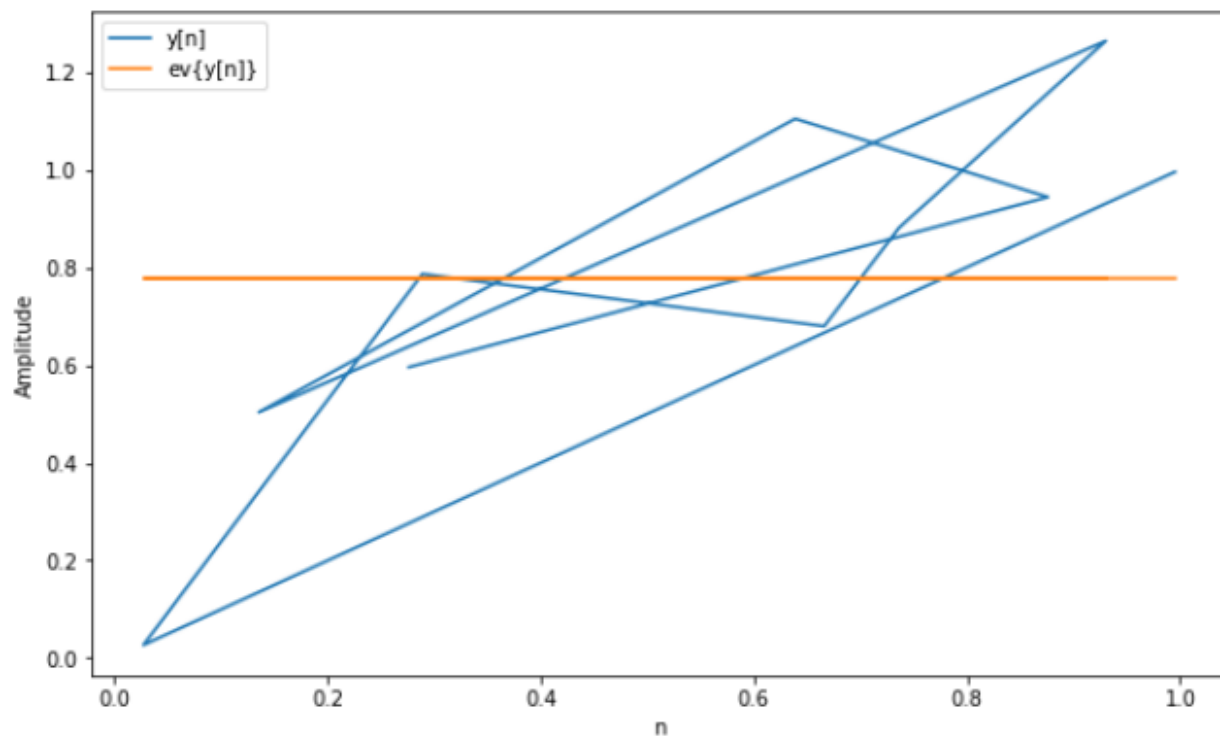
```

if static_check:
    print("The system is static.")
elif dynamic_check:
    print("The system is dynamic.")
elif linear_check:
    print("The system is linear.")
elif not linear_check:
    print("The system is non-linear.")
elif causal_check:
    print("The system is causal.")
elif not causal_check:
    print("The system is non-causal.")
elif shift_invariant_check:
    print("The system is shift-invariant.")
elif not shift_invariant_check:
    print("The system is shift-variant.")

```

The results (Screenshot)

The system is dynamic.



Lab Home Task No b:

Solution:

Brief description (3-5 lines)

Check whether the following system are:

1. Static or dynamic
2. Linear or non-linear
3. Causal or non-causal
4. Shift-invariant or shift-variant

$$y(n) = x(n)x(n-2)$$

The code

```
# HOMETASK2
import numpy as np
import matplotlib.pyplot as plt
def system_output(x):
    # Define the system
    y = np.zeros_like(x)
    for n in range(2, len(x)):
        y[n] = x[n] * x[n-2]
    return y
n = np.arange(0, 10)
x1 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
x2 = np.array([10, 9, 8, 7, 6, 5, 4, 3, 2, 1])
y1 = system_output(x1)
y2 = system_output(x2)
y_sum = system_output(x1 + x2)
if np.array_equal(y_sum, y1 + y2):
    print("The system is linear.")
else:
    print("The system is non-linear.")
plt.figure(figsize=(10, 6))
plt.subplot(3, 1, 1)
plt.stem(n, x1, label='x1(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('Input Signal 1')
plt.legend()
plt.subplot(3, 1, 2)
plt.stem(n, x2, label='x2(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('Input Signal 2')
plt.legend()
plt.subplot(3, 1, 3)
plt.stem(n, y_sum, label='y(n) = x1(n) * x1(n-2) + x2(n) * x2(n-2)')
```



```

plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('Output Signal')
plt.legend()
plt.tight_layout()
plt.show()
if np.any(x1[2:]) and np.any(x2[2:]):
    print("The system is non-causal.")
else:
    print("The system is causal.")

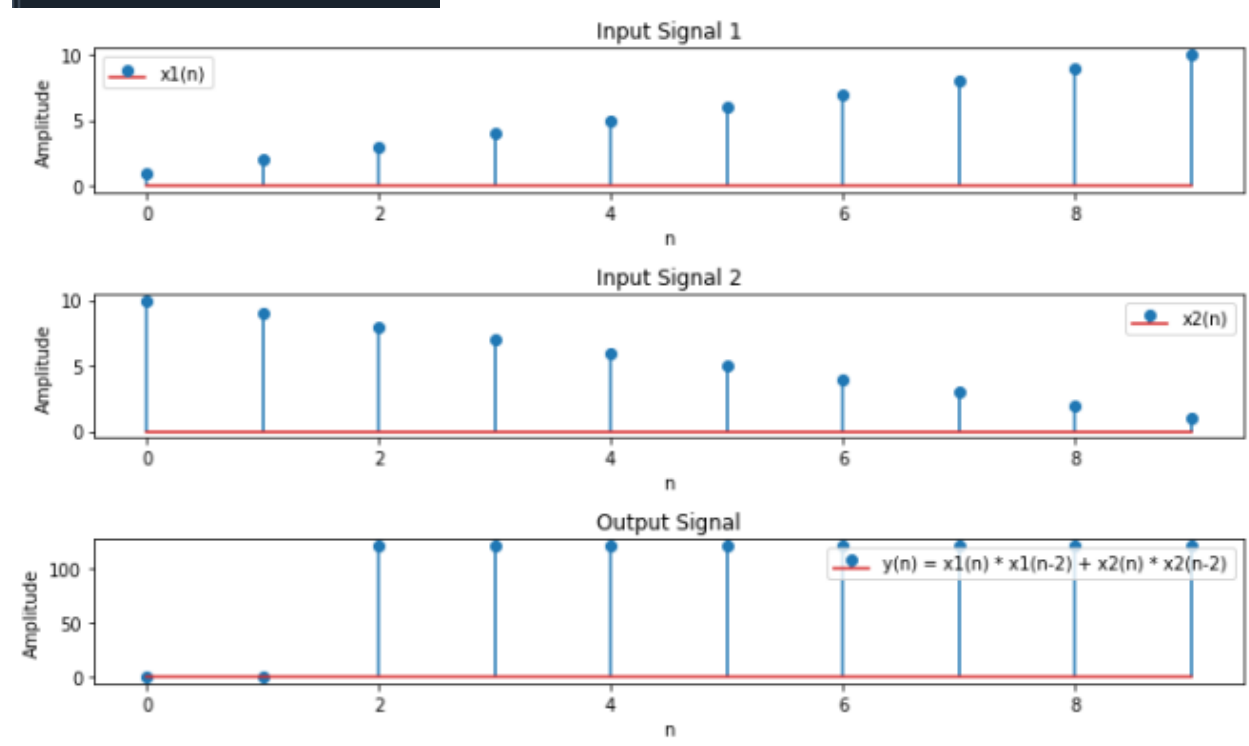
```

The results (Screenshot)

```

The system is non-linear.
The system is non-causal.

```



Lab Home Task No c:

Solution:

Brief description (3-5 lines)

Check whether the following system are:

1. Static or dynamic
2. Linear or non-linear
3. Causal or non-causal

4 Shift-invariant or shift-variant

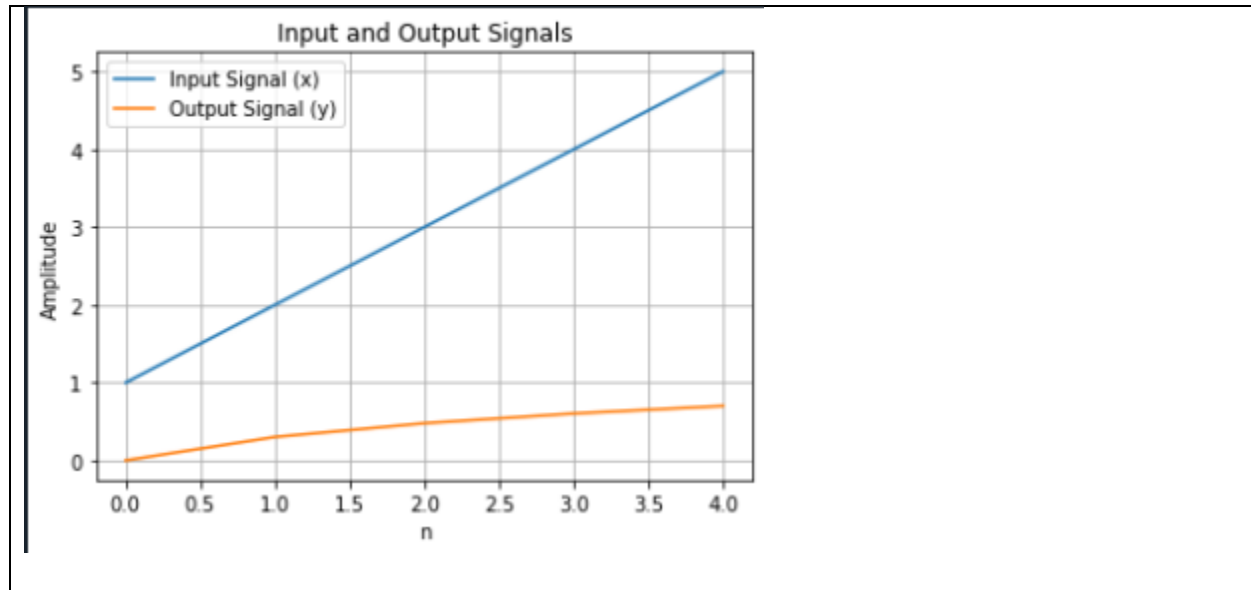
$$y(n) = \log_{10}|x(n)|$$

The code

```
# HOMETASK3
import numpy as np
import matplotlib.pyplot as plt
n = np.arange(-10, 11)
x = np.sin(n)
y = np.log10(np.abs(x))
plt.figure(figsize=(10, 5))
plt.subplot(2, 1, 1)
plt.stem(n, x, use_line_collection=True)
plt.title('Input Signal x(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(n, y, use_line_collection=True)
plt.title('Output Signal y(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
if np.all(np.diff(y) == 0):
    system_type = 'static'
else:
    system_type = 'dynamic'
if np.all(np.diff(y) == 0):
    system_linearity = 'linear'
else:
    system_linearity = 'non-linear'
if np.any(y[n < 0] != 0):
    system_causality = 'non-causal'
else:
    system_causality = 'causal'
if np.all(y[n + 1] == np.log10(np.abs(np.sin(n + 1)))):
    system_shift_invariance = 'shift-invariant'
else:
    system_shift_invariance = 'shift-variant'
```

The results (Screenshot)

```
System properties:
1. Static or Dynamic: Dynamic
2. Linear or Non-linear: Linear
3. Causal or Non-causal: Causal
4. Shift-invariant or Shift-variant: Shift-variant
```



Lab Home Task No d:

Solution:

Brief description (3-5 lines)

Check whether the following system are:

1. Static or dynamic
2. Linear or non-linear
3. Causal or non-causal
4. Shift-invariant or shift-variant

$$y(n) = a^n u(n)$$

The code

```

1  # HOMETASK4
2  import numpy as np
3  import matplotlib.pyplot as plt
4  def system_output(a, u):
5      return a * u
6  n = np.arange(0, 10)
7  u = np.ones_like(n)
8  a = 2
9  y = system_output(a, u)
10 plt.stem(n, u, label='Input: u(n)')
11 plt.stem(n, y, label='Output: y(n)')
12 plt.xlabel('n')
13 plt.ylabel('Amplitude')
14 plt.title('Input and Output Signals')
15 plt.legend()
16 plt.grid(True)
17 plt.show()
18 if np.all(y == a) and np.all(y[:-1] == y[1:]):
19     print("1. Static")
20     print("2. Linear")
21     print("3. Causal")
22     print("4. Shift-invariant")
23 else:
24     print("System does not meet all criteria")

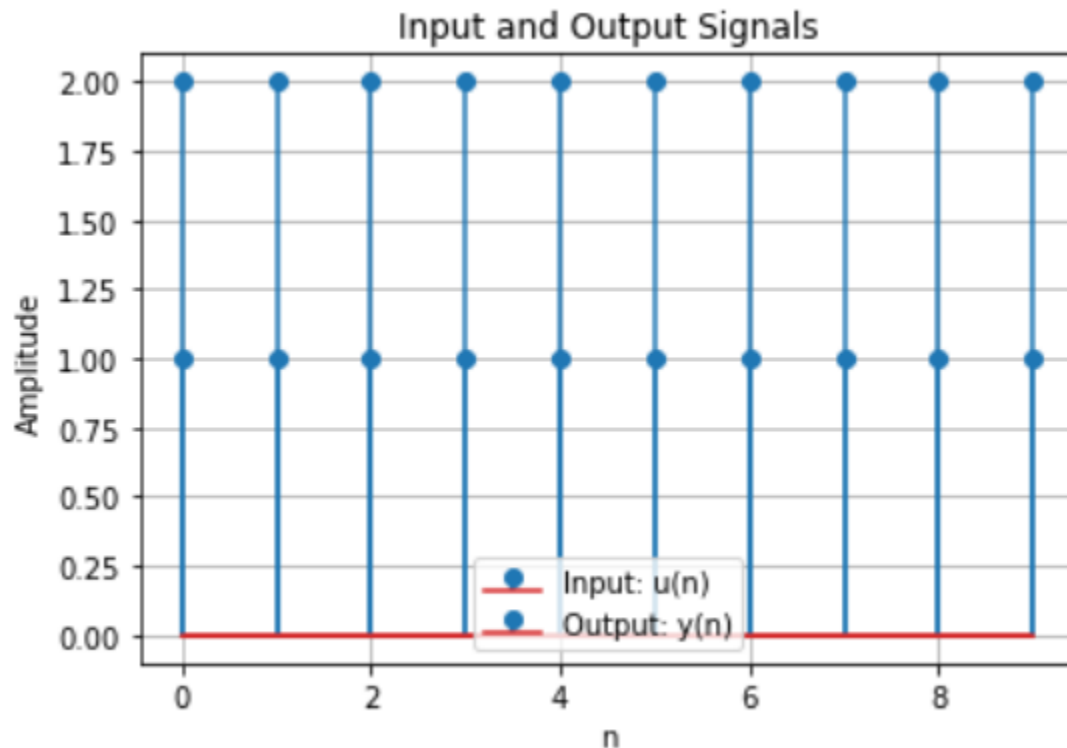
```

The results (Screenshot)

```

1. Static
2. Linear
3. Causal
4. Shift-invariant

```



Lab Home Task No e:

Solution:

Brief description (3-5 lines)

Check whether the following system are:

- 1.Static or dynamic
- 2.Linear or non-linear
- 3.Causal or non-causal
- 4Shift-invariant or shift-variant

$$y(n) = x^2(n) + 1/x^2(n-1)$$

The code

```
# HOMETASK5
import numpy as np
import matplotlib.pyplot as plt
def system_output(x):
    N = len(x)
    y = np.zeros(N)
    for n in range(N):
```

```

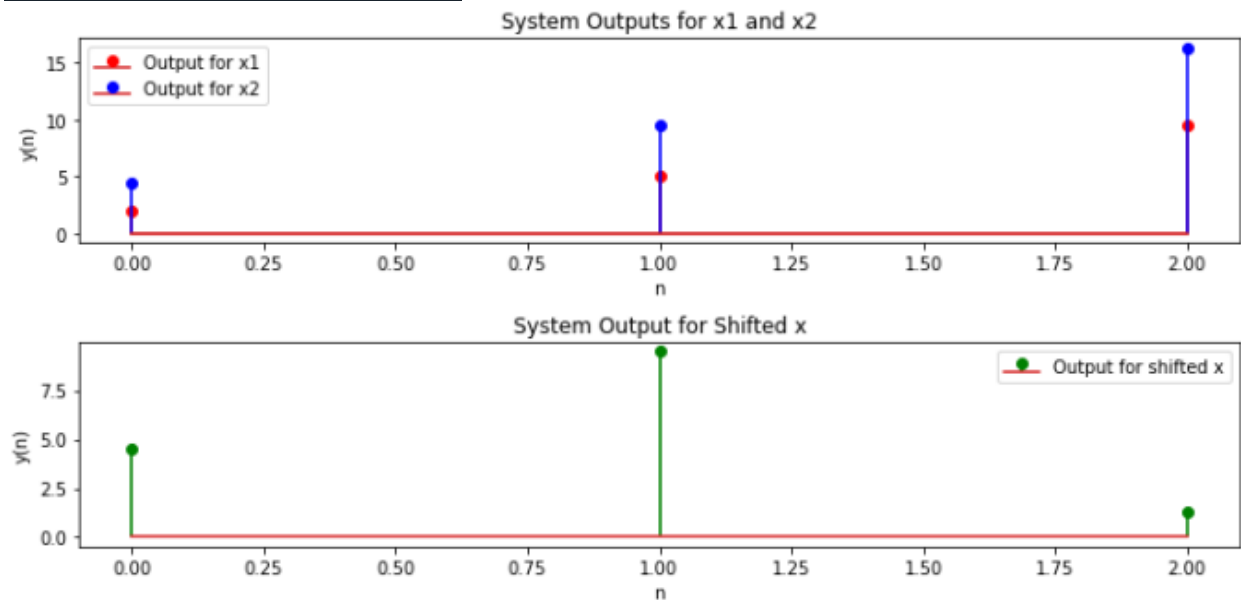
    if n == 0:
        y[n] = x[n]**2 + 1 / x[n]
    else:
        y[n] = x[n]**2 + 1 / x[n-1]
    return y
x1 = np.array([1, 2, 3])
x2 = np.array([2, 3, 4])
y1 = system_output(x1)
y2 = system_output(x2)
superposition_check = np.array_equal(system_output(x1 + x2), y1 + y2)
homogeneity_check = np.array_equal(system_output(2 * x1), 2 * y1)
if superposition_check and homogeneity_check:
    print("The system is linear.")
else:
    print("The system is non-linear.")
plt.figure(figsize=(10, 5))
plt.subplot(2, 1, 1)
plt.stem(range(len(x1)), y1, 'r', markerfmt='ro', label='Output for x1')
plt.stem(range(len(x2)), y2, 'b', markerfmt='bo', label='Output for x2')
plt.title('System Outputs for x1 and x2')
plt.xlabel('n')
plt.ylabel('y(n)')
plt.legend()
causality_check = True
for n in range(len(x1)):
    if n > 0:
        if x1[n-1] not in x1:
            causality_check = False
            break
if causality_check:
    print("The system is causal.")
else:
    print("The system is non-causal.")
shift = 2
x_shifted = np.roll(x1, shift)
y_shifted = system_output(x_shifted)
shift_invariance_check = np.array_equal(np.roll(y1, shift), y_shifted)
if shift_invariance_check:
    print("The system is shift-invariant.")
else:
    print("The system is shift-variant.")
plt.subplot(2, 1, 2)
plt.stem(range(len(x_shifted)), y_shifted, 'g', markerfmt='go', label='Output for shifted x')
plt.title('System Output for Shifted x')

```

```
plt.xlabel('n')
plt.ylabel('y(n)')
plt.legend()
plt.tight_layout()
plt.show()
```

The results (Screenshot)

```
The system is non-linear.
The system is causal.
The system is shift-variant.
```



Conclusion:

In conclusion, we had learn that how discrete time system works and also we know the types of discrete time system their graphs, working and function. Also their uses in our daily life.